

بسمه تعالی

گزارش کار پروژه درس داده‌کاوی

پیاده‌سازی، تنظیم پارامتر و مقایسه الگوریتم‌های
(KNN) نزدیک‌ترین همسایه K و (Decision Tree) درخت تصمیم

Mobile Price Classification: دیتاست

بهزاد رادمنشی

استاد: مرتضی بهرامی

نیمسال دوم سال تحصیلی 1404-1405

لینک مخزن GitHub: <https://github.com/Behzad-radmaneshi83/-mobile-price-decision-tree-knn>

مقدمه و معرفی پروژه ۱.

نزدیکترین K و (Decision Tree) هدف این پروژه، آشنایی عملی با دو الگوریتم پایه‌ی طبقه‌بندی یعنی درخت تصمیم و همچنین نرم‌افزار Python است. در این پروژه هر دو مدل با استفاده از زبان برنامه‌نویسی (KNN) همسایه پیاده‌سازی شده‌اند، ابرپارامترهای هر کدام در بازه‌های مشخص تغییر داده شده و عملکرد مدل‌ها با معیارهای RapidMiner استاندارد ارزیابی طبقه‌بندی مقایسه شده است.

معرفی دیتاست ۱.۱

است که هدف آن پیش‌بینی بازه‌ی قیمت گوشی‌های موبایل «Mobile Price Classification»، دیتاست مورد استفاده ظرفیت باتری، وضوح صفحه نمایش، وجود RAM، چهار کلاس: 0 تا 3 (بر اساس مشخصات فنی دستگاه مانند حافظه نام دارد و مسئله از نوع price_range بلوتوث/وای‌فای/جی‌پی‌اس و سایر ویژگی‌هاست. ستون هدف در این دیتاست است (Multi-class Classification) طبقه‌بندی چندکلاسه.

معرفی مدل‌ها ۲.

(Decision Tree) درخت تصمیم ۲.۱

درخت تصمیم یک مدل طبقه‌بندی مبتنی بر ساختار درختی است که با تقسیم پیاپی فضای ویژگی‌ها بر اساس معیارهایی مانند Gini Index یا Information Gain (Entropy)، عمق درخت، تأثیر مستقیمی بر عملکرد مدل دارد K داده‌های آموزش تعیین می‌کند. انتخاب مقدار مهم‌ترین ابرپارامتر کنترل‌کننده‌ی پیچیدگی مدل است (max_depth).

(KNN) نزدیکترین همسایه K ۲.۲

نمونه‌ی نزدیک آن در K یک الگوریتم مبتنی بر فاصله است که برچسب هر نمونه‌ی جدید را بر اساس رأی اکثریت KNN و نوع معیار فاصله (افلیدسی یا منهتن (تأثیر مستقیمی بر عملکرد مدل دارد K داده‌های آموزش تعیین می‌کند. انتخاب مقدار به مقیاس ویژگی‌ها حساس است، نرمال‌سازی داده‌ها پیش از آموزش این مدل ضروری است KNN از آنجا که

روش انجام کار ۳.

Python پیاده‌سازی با ۳.۱

(روی ستون هدف stratify با train_test_split) داده‌ها با نسبت ۸۰ به ۲۰ به دو بخش آموزش و آزمون تقسیم شدند، نرمال‌سازی شدند؛ درخت تصمیم به دلیل عدم حساسیت به مقیاس داده StandardScaler ویژگی‌ها با KNN برای مدل، در عمق‌های مختلف (۳، ۵، ۷، entropy و gini) بدون نرمال‌سازی آموزش داده شد. برای درخت تصمیم، معیار تقسیم با دو معیار فاصله‌ی اقلیدسی و منهتن (K = 3, 5, 7, 9, 11, 15) تعداد همسایه‌ها، KNN و بدون محدودیت (و برای ۱۰ آزمایش شدند.

RapidMiner پیاده‌سازی با ۳.۲

K-NN و Split Data، دیتاست مشابه بارگذاری و با استفاده از عملگرهای RapidMiner، در محیط و information_gain و gini_index بین criterion فرآیند آموزش و آزمون طراحی شد. برای درخت تصمیم پارامتر، در مقادیر ۱، ۳، ۵، k نیز پارامتر KNN در مقادیر ۱، ۳، ۵، ۷ و ۱۰ تغییر داده شد. برای maximal_depth پارامتر Performance بررسی شد. عملکرد هر اجرا با عملگر Manhattan و Euclidean و ۹ با دو معیار فاصله‌ی ۷ ثبت شد (F1 و Accuracy، Precision، Recall شامل).

۴. نتایج

۴.۱ — نتایج درخت تصمیم Python

F1-Score	Recall	Precision	Accuracy	عمق (max_depth)	معیار (criterion)
0.7490	0.7450	0.7654	74.50%	3	gini
0.8317	0.8300	0.8349	83.00%	5	gini
0.8327	0.8325	0.8343	83.25%	7	gini
0.8208	0.8200	0.8241	82.00%	10	gini
0.8302	0.8300	0.8319	83.00%	بدون محدودیت	gini
0.7537	0.7525	0.7617	75.25%	3	entropy
0.8720	0.8725	0.8747	87.25%	5	entropy
0.8715	0.8725	0.8722	87.25%	7	entropy
0.8789	0.8800	0.8788	88.00%	10	entropy
0.8692	0.8700	0.8696	87.00%	بدون محدودیت	entropy

برابر با **F1-Score 0.8789** با دقت **88.00%** و **max_depth = 10** و **criterion = entropy**: بهترین اجرا

۴.۲ — نتایج درخت تصمیم RapidMiner

Accuracy (information_gain)	Accuracy (gini_index)	عمق (max_depth)
25.00%	25.00%	1
73.43%	75.21%	3
77.29%	78.50%	5
82.14%	80.21%	7
82.93%	75.07%	10

با دقت **82.93%** و **max_depth = 10** و **criterion = information_gain**: بهترین اجرا

۴.۳ KNN — Python نتایج

F1-Score	Recall	Precision	Accuracy	K	معیار فاصله
0.4718	0.4725	0.4911	47.25%	3	Euclidean
0.5054	0.5000	0.5211	50.00%	5	Euclidean
0.5199	0.5175	0.5267	51.75%	7	Euclidean
0.5674	0.5625	0.5753	56.25%	9	Euclidean
0.5484	0.5450	0.5566	54.50%	11	Euclidean
0.5757	0.5725	0.5828	57.25%	15	Euclidean
0.4782	0.4800	0.5018	48.00%	3	Manhattan
0.5605	0.5600	0.5708	56.00%	5	Manhattan
0.5733	0.5750	0.5800	57.50%	7	Manhattan

F1-Score	Recall	Precision	Accuracy	K	معیار فاصله
0.5760	0.5750	0.5824	57.50%	9	Manhattan
0.6033	0.6025	0.6093	60.25%	11	Manhattan
0.6443	0.6475	0.6448	64.75%	15	Manhattan

برابر با F1-Score 0.6443 با دقت 64.75% و $K = 15$ و Manhattan بهترین اجرا: معیار فاصله

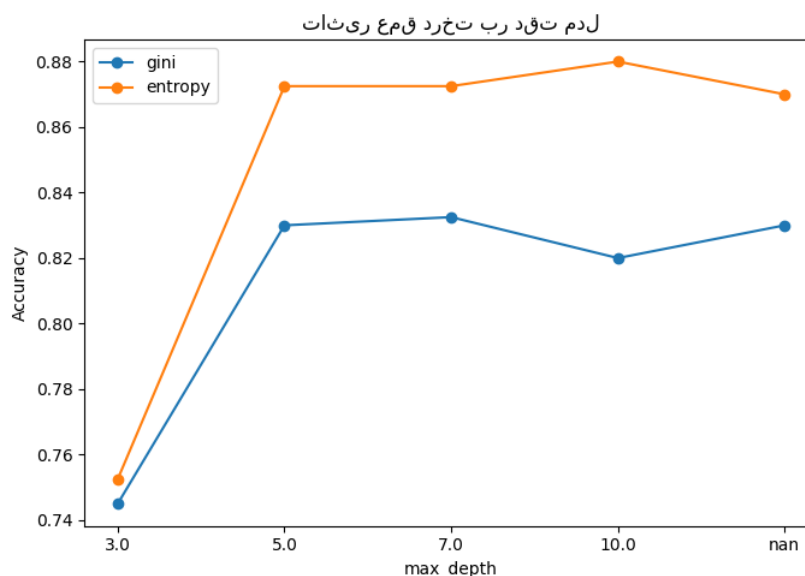
۴.۴ KNN — RapidMiner نتایج

Accuracy (Manhattan)	Accuracy (Euclidean)	K
100.00%	100.00%	1
98.14%	98.00%	3
95.14%	94.93%	5
94.43%	94.64%	7
—	94.71%	9

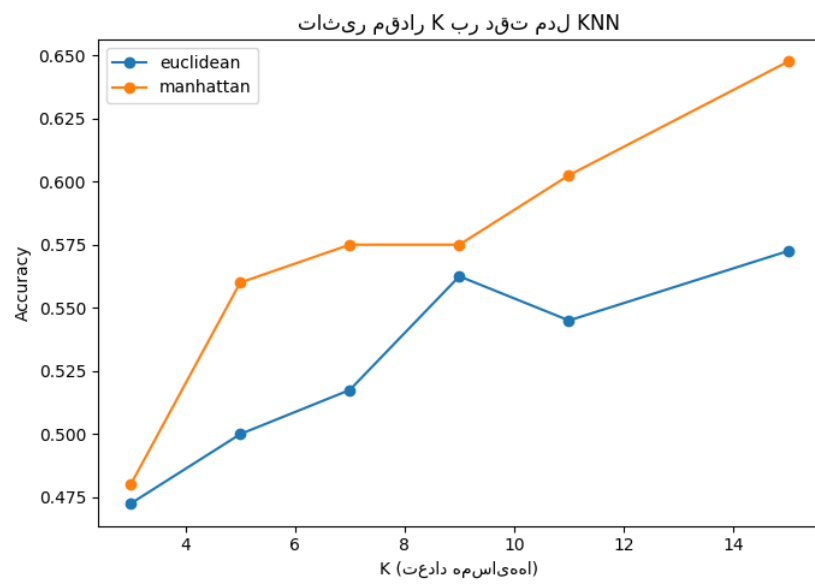
در داده‌های دریافتی ثبت نشده است Manhattan با معیار $K=9$ برای Accuracy مقدار *

مدل عملاً هر نمونه را به نزدیک‌ترین همسایه‌ی $K=1$ باید با احتیاط تفسیر شود؛ در $K=1$ نکته مهم: دقت 100% در خودش (که می‌تواند نمونه‌ی بسیار مشابه یا حتی خودِ داده در صورت آموزش/آزمون همپوشان باشد) نسبت می‌دهد و این باشد نه لزوماً برتری واقعی مدل (Train و Test مثلاً نشت داده بین) می‌تواند نشانه‌ی ارزیابی نامناسب داده

۴.۵ نمودارهای تحلیلی

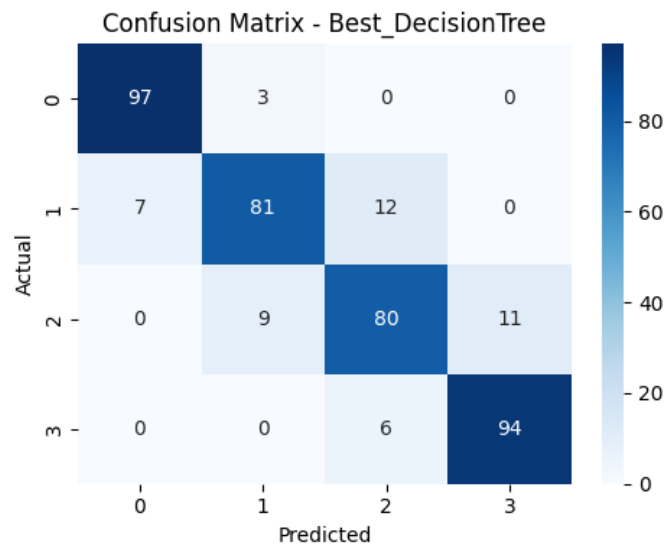


(Python) شکل ۱: تأثیر عمق درخت تصمیم بر دقت مدل

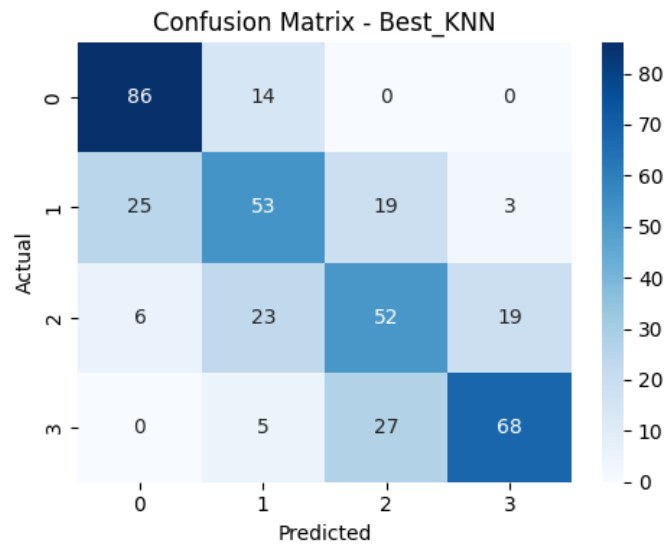


KNN (Python) بر دقت مدل K شکل ۲ نتایج مقدار

۴.۶ (Python) ماتریس درهم‌ریختگی بهترین مدل‌ها



شکل ۳: ماتریس درهم‌ریختگی بهترین مدل درخت تصمیم ($entropy, depth=10$)



شکل ۴: ماتریس درهم‌ریختگی بهترین مدل KNN (Manhattan, $K=15$)

در ماتریس درهم‌ریختگی درخت تصمیم، اعداد قطر اصلی (پیش‌بینی صحیح) به‌طور محسوسی بزرگ‌تر و خطاهای خارج پراکندگی خطا بین کلاس‌های مجاور (مثلاً کلاس ۱ و ۲) (بیشتر دیده KNN از قطر کمتر است، در حالی‌که در ماتریس می‌شود.

تحلیل و پاسخ به سؤالات پروژه ۵.

Overfitting تأثیر تغییر پارامترهای درخت تصمیم بر دقت و ۵.۱

دقت، (مثلاً) با افزایش عمق درخت از مقادیر بسیار کم، (Python و RapidMiner) در هر دو پیاده‌سازی دقت تنها 25.00% است که تقریباً معادل حدس تصادفی $depth=1$ با RapidMiner به‌شدت پایین است؛ برای مثال در (کم‌پرازش (شدید است، چون درخت آن‌قدر ساده است که نمی‌تواند الگوهای Underfitting بین چهار کلاس بوده و نشانه‌ی داده را یاد بگیرد.

gini با معیار Python با افزایش عمق، دقت به‌طور پیوسته رشد می‌کند تا به یک نقطه‌ی بهینه برسد. در پیاده‌سازی دقت کمی افت کرد (82.00%) که نشانه‌ی شروع $depth=10$ به دست آمد و در $depth=7$ (83.25%) بهترین دقت در (بیش‌پرازش (است؛ مدل در این حالت به‌جای یادگیری الگوی کلی، جزئیات نویزدار داده‌های آموزش را حفظ Overfitting برای معیار RapidMiner می‌کند و همین باعث افت تعمیم‌پذیری روی داده‌ی آزمون می‌شود. رفتار مشابهی در $depth=10$ به اوج (80.21%) رسید و در $depth=7$ دیده شد: دقت در gini_index به 75.07% افت کرد.

هم‌چنان رشد داشت $depth=10$ در هر دو پیاده‌سازی، دقت تا entropy/information_gain در مقابل، با معیار و نشانه‌ی واضحی از افت دقت ($depth=10$ در RapidMiner: 82.93%؛ $depth=10$ در Python: 88.00%) در این بازه‌ی عمق دیده نشد؛ هرچند به‌طور کلی افزایش بی‌رویه‌ی عمق درخت، بدون هرس (آشکار Overfitting یعنی) یا محدودیت‌های دیگر، همواره ریسک بیش‌پرازش را افزایش می‌دهد (Pruning).

K و بهترین مقدار KNN در K تأثیر تغییر ۵.۲

Manhattan از 3 به 15، دقت مدل به‌طور کلی روند صعودی داشت (برای معیار K با افزایش Python، در پیاده‌سازی باعث حساسیت مدل به K رسید)، که نشان می‌دهد مقادیر بسیار کوچک $K=15$ به 64.75% در $K=3$ از 48.00% در، رأی‌گیری را پایدارتر می‌کند. بر اساس این نتایج K در حالی که افزایش (محلی Overfitting) نویز داده‌ی آموزش می‌شود بود (Manhattan با معیار) برابر 15 Python در K بهترین مقدار.

دقت کاهش یافت، K ثابت شد و با افزایش (100%) $K=1$ روند برعکس مشاهده شد: بالاترین دقت در RapidMiner در این تفاوت رفتار قابل توجه است و به احتمال زیاد ناشی از تفاوت در نحوه‌ی ($K=9$ و $K=7$ تا حدود 94-95% در) با توجه به KNN است، نه یک ویژگی ذاتی الگوریتم RapidMiner پیش‌پردازش، نرمال‌سازی یا اعتبارسنجی داده در به‌عنوان انتخاب قابل‌اتکا برای تعمیم‌پذیری واقعی توصیه نمی‌شود؛ در $K=1$ مقدار، ریسک بیش‌پرازش/نشت داده در بین 5 تا 9 گزینه‌ی معقول‌تری است K، RapidMiner، صورت نیاز به یک مقدار پایدارتر در.

KNN مقایسه کلی درخت تصمیم و ۵.۳

داشت (بهترین دقت درخت تصمیم KNN درخت تصمیم به‌وضوح عملکرد بهتری نسبت به Python، در پیاده‌سازی این تفاوت را می‌توان به ماهیت ویژگی‌های دیتاست نسبت داد: بسیاری (KNN 64.75%) در برابر بهترین دقت 88.00% رابطه‌ی تقریباً آستانه‌ای/پلکانی با کلاس هدف (مانند ram) Mobile Price Classification از ویژگی‌های دیتاست به فاصله‌ی اقلیدسی/منهتن KNN دارند که برای مدل‌های مبتنی بر تقسیم مانند درخت تصمیم بسیار مناسب است، در حالی‌که در فضای همگی ویژگی‌ها حساس است و ویژگی‌های کم‌اهمیت یا مقیاس‌نشده می‌توانند فاصله‌ها را مخدوش کنند. دقت بسیار بالاتری (تا 94-100%) نسبت به درخت تصمیم (حداکثر KNN: نتیجه‌ی متفاوتی دیده شد RapidMiner در همخوانی ندارد، به این نتیجه با احتیاط باید نگاه کرد Python نشان داد. با توجه به این‌که این تفاوت با نتایج (82.93% (به بخش ۵.۴ مراجعه شود).

Python و RapidMiner مقایسه نتایج ۵.۴

با پارامترهای مشابه، تفاوت معناداری مشاهده شد، به‌خصوص Python و RapidMiner، بین نتایج به‌دست‌آمده از KNN برای مدل:

- برای درخت تصمیم، روند کلی (بهبود دقت با افزایش عمق تا حد معینی (در هر دو پیاده‌سازی مشابه بود، هرچند مقادیر (برابر 82.93% بود RapidMiner برابر 88.00% و بهترین دقت Python مثلاً بهترین دقت) دقیق دقت متفاوت بودند.
- بین 94% تا RapidMiner بین 47% تا 65% و در Python تفاوت بسیار چشمگیرتر بود: دقت در KNN، برای بود. این اختلاف زیاد به احتمال قوی ناشی از تفاوت در پیش‌پردازش (مانند نوع و ترتیب نرمال‌سازی، نحوه‌ی تقسیم 100%

است، نه برتری ذاتی (RapidMiner) یا احتمال وجود همپوشانی بین داده‌ی آموزش و آزمون در فرآیند Train/Test، KNN در اجرای الگوریتم RapidMiner نرم‌افزار

نتیجه‌گیری این بخش آن است که مقایسه‌ی مستقیم بین دو محیط تنها زمانی معتبر است که تنظیمات پیش‌پردازش (به‌ویژه نرمال‌سازی و روش تقسیم داده (بین دو پیاده‌سازی کاملاً یکسان‌سازی شده باشد؛ در غیر این صورت اختلاف دقت می‌تواند ناشی از تفاوت محیط اجرا باشد نه تفاوت الگوریتم

۶. نتیجه‌گیری

در دو محیط Mobile Price Classification روی دیتاست KNN در این پروژه دو الگوریتم طبقه‌بندی درخت تصمیم و K پیاده‌سازی و با تغییر ابرپارامترهای کلیدی (عمق و معیار تقسیم برای درخت تصمیم؛ مقدار Python و RapidMiner مورد ارزیابی قرار گرفتند. نتایج نشان داد که درخت تصمیم به‌طور کلی برای این دیتاست (KNN و معیار فاصله برای و عمق متوسط تا بالا) ۷ تا ۱۰ (entropy/information_gain انتخاب مناسب‌تر و پایدارتری است، به‌ویژه با معیار نیازمند هماهنگی دقیق تنظیمات پیش‌پردازش Python و RapidMiner همچنین مشخص شد که مقایسه‌ی مستقیم بین است، در غیر این صورت نتایج قابل مقایسه نخواهند بود

۷. پیوست

لینک مخزن GitHub پروژه: <https://github.com/Behzad-radmaneshi83/-mobile-price-decision-tree-knn>

RapidMiner فایل فرآیند (mobile_price_classification.py) فایل‌های تحویلی همراه این گزارش: کد پایتون و تصاویر نمودار ها/ماتریس‌های درهم‌ریختگی (all_results_summary.csv) جدول کامل نتایج (all_results_summary.csv)، (all_results_summary.csv) (all_results_summary.csv) (all_results_summary.csv)